

ROLLING-HORIZON FEASIBLE OPERATING REGION

Parallel directional sweep

Making the per-transformer rolling FOR 1.82× faster:
what was changed, what it costs, and why the answers still hold

August 2026 · Campaign report

In one paragraph. Each period of the rolling FOR solves 12 sweep directions. They used to be solved one after another. They are now solved several at a time, in separate CPLEX sessions. Nothing about the physics changed: same exact quadratic form, same constraints, same switches. The committed dispatch comes out **bit-identical**. Measured end to end on all nine transformers under one controlled machine base, the campaign runs in **19.6 h** instead of **35.7 h**.

1 The headline

Whole per-transformer campaign	1.82×
Best case (TR9, 90 batteries)	2.73×
Worst case (TR7, 259 batteries)	1.31×
Solves	5184 of 5184 Optimal
Committed dispatch (SOC files)	bit-identical, all nine
FOR boundary vertices	within 3 W–70 W

The committed dispatch is the quantity the papers use. It is unchanged, exactly. Only the FOR boundary vertices move, and by amounts far below the model’s own feasibility tolerance. Section 5 gives the numbers. These are measurements, not projections. Every run was made one transformer per fresh AIMMS session, on a machine held in a fixed state.

2 How it works

Within one period the 12 directions share the same feasible region. The data reset is deterministic. The rolling map is fixed. Slot-1 dispatch is frozen. And $\text{FOR_TieEps} = 0$ makes each vertex independent of solve history. So the directions may be solved in any order, or at the same time.

Each direction receives its own copy of the period’s generated matrix and its own solver session. Two settings control the split:

- RollAsyncK — how many directions solve at once (sessions);
- RollAsyncThreads — threads per session.

The machine has 24 physical cores, so $K \times \text{threads} = 24$ is the budget.

Two derived quantities explain every result below:

- **inflation** — async mean solve divided by sequential mean solve: what concurrency costs each individual solve;
- **concurrency** — total solver seconds divided by wall seconds: how many solves really overlapped.

Net speed-up is approximately concurrency divided by inflation. Neither number alone predicts anything.

3 Results — all nine transformers

	batt	split	seq.	base	speed-up	infl.	conc.
TR4	46	6×4	35.7	18.6	1.92×	2.20	3.63
TR3	49	6×4	39.1	20.1	1.95×	2.33	3.95
TR1	77	6×4	96.5	50.1	1.93×	2.31	3.84
TR9	90	6×4	195.8	71.8	2.73×	1.59	3.78
TR2	95	6×4	140.4	71.4	1.97×	2.21	3.79
TR8	124	4×6	298.9	125.1	2.39×	1.18	2.42
TR5	214	4×6	516.5	204.4	2.53×	1.11	2.39
TR7	259	4×6	411.2	314.0	1.31×	1.97	2.17
TR6	262	4×6	409.1	301.6	1.36×	1.99	2.20
			2143.2	1177.0	1.82×		

Wall clock in minutes.

3.1 The gain is not monotonic in size

It peaks in the middle. TR5, at 214 batteries, beats every small transformer except TR9; then the gain collapses above about 250. Three regimes, not a trend:

band	behaviour
below ~120 batteries	~1.9×; the split is neutral (Section 4)
124–214	best — 2.4–2.5×, inflation only 1.11–1.18
above ~250	ceiling ~1.35× — inflation ~2.0 and no split fixes it

TR7 and TR6 are the same result twice: 259 and 262 batteries, inflation 1.97 and 1.99, concurrency 2.17 and 2.20, speed-up 1.31× and 1.36×. The ceiling is structural, not a bad night. It is not memory exhaustion either — peak AIMMS 14.8 GB with 8.1 GB still free.

3.2 Why TR9 tops the table

TR9 and TR2 are nearly the same run: 71.8 against 71.4 min, 28.29 against 28.14 s per solve, concurrency 3.78 against 3.79. The whole difference in speed-up comes from the *sequential* side, where TR9’s solves are intrinsically 40 % heavier (17.76 against 12.73 s). Heavier solves carry the parallel overhead better.

Worth remembering when quoting these: the speed-up column is partly a statement about the sequential baseline, not only about the parallel gain.

4 Choosing the split

Three experiments. **TR4 tuning** (46 batteries, full runs): sequential 35m20s, 6×2 28m16s, **6×4 20m13s**, 12×2 24m39s. Above about six sessions the machine saturates.

TR6 probe (262 batteries, two periods per split, 24 threads throughout):

split	wall	mean solve	concurrency
6 × 4	692 s	95.4 s	3.31
4 × 6	621 s	60.3 s	2.33
3 × 8	706 s	53.4 s	1.82
2 × 12	746 s	39.3 s	1.27

Fewer and fatter sessions cut the time per solve sharply, from 95 s to 39 s, but the concurrency achieved is only $0.55\text{--}0.63 \times K$ whatever the split.

TR3 split test (49 batteries, full runs): 6×4 20.05 min, 4×6 20.02 min. A dead heat. The 4×6 split cut the mean solve from 8.24 to 5.44 s exactly as predicted, and concurrency fell from 3.95 to 2.61, cancelling it out.

Judge a split by wall clock, never by mean solve time. On TR6, 2×12 had the best mean solve of any split and the worst wall clock. On TR3, 4×6 improved the mean by a third and moved the wall by two seconds.

So the crossover is *not* “6×4 is better below 120 batteries”. It is:

- below ~120 — the split does not matter; 6×4 is kept because there is no reason to change;
- above ~120 — 4×6, and the margin is large;
- above ~250 — 4×6 is still the best available, but nothing recovers the ceiling.

4.1 How the probe held up

The TR6 probe predicted inflation of 1.73 for 4×6; the full 48-period run gave 1.99, and the wall clock came in at 5.03 h against a 4.5 h projection. Directionally right, quantitatively soft — good enough to pick a split, not to promise a wall clock.

5 The batch wrapper cost 1.48× for nothing

RunFOR_Rolling_PerTrafo_Async used to load the whole feeder once at the start, only to read the list of transformer tags. But RunFOR_Rolling empties everything and reloads as its first action. So that load bought nothing, while it grew the AIMMS heap to full-feeder size for the rest of the session.

	standalone	in the batch	penalty
TR4	7.76 s/solve	11.58 s/solve	1.49×
TR6	95.4 s/solve	141.0 s/solve	1.48×

The same penalty appears at different positions in the batch, so it was a constant and not creeping fragmentation. Removing the load fixed it: TR4 went from 40.8 min to 18.9 min.

This is why the campaign of 17–18 August is not quoted here. It ran across a batch, across days and across machine loads, and produced two transformers *slower* than sequential. The table in Section 3 replaces it entirely.

6 Is it still correct?

Yes, with one caveat.

- **Every solve Optimal** — 5184 of 5184 across the nine runs.
- **Committed dispatch bit-identical** — all nine SOC files match their sequential twins byte for byte.
- **Async is deterministic run to run.** Five transformers were compared against their August async runs and came back at exactly zero, including TR9 across two days and a reboot. The drift below is against the *sequential* solver only.
- **FOR vertices drift a little, and the drift grows with size.**

	$\max \Delta \text{proj} $	rows over gate (of 576)
TR3, TR4	3×10^{-6} p.u. (3 W)	0
TR1, TR2, TR9	7×10^{-6} – 9×10^{-6} p.u.	6–10
TR8	2.2×10^{-5} p.u.	0
TR5	3.9×10^{-5} p.u.	0
TR6	4.8×10^{-5} p.u.	0
TR7	7×10^{-5} p.u. (70 W)	1

The cause is the thread cap changing the order of arithmetic inside the barrier solver. For scale, 70 W is about **14 times inside** the model’s own Feasibility_Tolerance of 1×10^{-3} p.u. = 1 kW, and some 0.005 % of the extent of the region.

Caveat. If a number needs the strict 5×10^{-6} gate, run that case sequentially. The sequential `_gmp` results stay the set to quote from.

The invariant is `proj`, not `P` and `Q` on their own. Flat faces of the region make the argmax a segment, so `P` and `Q` can slide while `proj` holds.

7 What to run

goal	procedure
One transformer, controlled conditions	RunTRn_Base — the split is already correct for its size
All nine in one pass	RunFOR_Rolling_PerTrafo_Async — fixed wrapper, size-adaptive split, writes <code>_asyncB</code>
Re-probe after a hardware change	RunTR6_AsyncTune — ~1.5 h, four splits, two periods each

The older RunTRn_Async wrappers are all hard-coded 6×4, so **TR5–TR8 are mis-tuned there** — prefer RunTRn_Base. RunTR6_Async38 and RunTR3_Base4x6 are kept as records of their experiments; neither split won.

Rules of thumb on this 24-core, 32 GB machine:

- keep $K \times \text{threads} = 24$;

- 4×6 above ~120 batteries; below that the split is neutral and 6×4 is the default;
- not at full-feeder scale *on this machine* (`ReduceNetwork = 0`) — even the sequential run exhausted its 32 GB here. The same run completed elsewhere in 82.17 h, 576/576 optimal, so the wall is local rather than a property of the model;
- when re-tuning, pick the lowest *wall clock*;
- one transformer per fresh AIMMS session if the timings are to be compared.

Memory observed, peak AIMMS against minimum free: TR4 3.4 / 21.5, TR9 7.2 / 17.3, TR5 11.2 / 11.8, TR7 14.8 / 8.1, TR6 14.4 / 9.1 GB.

8 Files

file	what
README.md	this report, in markdown
timings.csv	every transformer: size, all three passes, speed-up, inflation, concurrency, validation
split_tests.csv	all three split experiments in one table
BASELINE_PROTOCOL.md	how the controlled base was set up and run
baseline/	the campaign’s environment log and per-run timings
probe/	raw TR6 probe output
src/	L ^A T _E X source of this PDF

Full FOR results stay at the repository root as `FOR_rolling_<tag>_{gmp,base,async,asyncB}.csv`, with the `_soc_` and `_fair_` variants. The model procedures are in `MainProject/OPF.ams`, and the snapshots taken before each change are the `OPF_pre_*.ams.txt` backups beside it.